

Sandbox Lab 2 — Submission

AI-Assisted Code Refactoring: Strategy & Factory Pattern

1. Original Code — Monolithic Implementation

The original `handleQuizAnswer` function combined scoring logic, UI rendering, and `localStorage` writes into a single monolithic switch statement — making it hard to test, extend, or reuse across environments.

HANDLEQUIZANSWER.JS — LEGACY IMPLEMENTATION

```
function handleQuizAnswer(questionData, userResponse) {
  let score = 0;
  let isCorrect = false;

  // MONOLITHIC SWITCH - scoring + UI + storage all coupled
  switch (questionData.type) {
    case 'MULTIPLE_CHOICE':
      console.log('Processing MCQ UI layout...'); // UI logic mixed in
      if (questionData.correctAnswer === userResponse) {
        score = questionData.points;
        isCorrect = true;
      }
      break;

    case 'TRUE_FALSE':
      const formattedResponse = String(userResponse).toLowerCase();
      if (formattedResponse === questionData.correctAnswer) {
        score = questionData.points; isCorrect = true;
      }
      break;

    case 'DRAG_AND_DROP':
      let matches = 0;
      questionData.correctOrder.forEach((item, index) => {
        if (userResponse[index] === item) matches++;
      });
      score = (matches === questionData.correctOrder.length)
        ? questionData.points
        : (matches / questionData.correctOrder.length) * questionData.points;
      isCorrect = matches === questionData.correctOrder.length;
      break;

    case 'SHORT_ANSWER':
      const sanitizedInput = userResponse.trim().toLowerCase();
      if (questionData.acceptedAnswers.includes(sanitizedInput)) {
        score = questionData.points; isCorrect = true;
      }
      break;

    default:
      console.error('Unknown question type: ' + questionData.type);
      return;
  }
}
```

```

    }

    // STORAGE LOGIC MIXED IN
    localStorage.setItem('last_score', score);

    // UI FEEDBACK LOGIC MIXED IN
    document.getElementById('feedback-box').innerText =
      isCorrect ? 'Correct! Well done.' : 'Try again.';
    document.getElementById('feedback-box').style.color =
      isCorrect ? '#007600' : '#B12704';

    return { questionId: questionData.id, score, status: isCorrect ? 'completed' :
'failed',
      timestamp: new Date().toISOString() };
  }
}

```

2. AI Prompting — Iterative Refinement

Rather than writing a single prescriptive prompt, the refactoring was developed through three iterations. Each prompt built on what the previous output got wrong.

Prompt 1 — First Attempt (Too Broad)

```

"Refactor this handleQuizAnswer function to be cleaner and follow SOLID
principles."

```

Result: Copilot split the function into smaller helpers but kept all logic in a single file with the same coupling. No real architectural change — just formatting.

Prompt 2 — Refined (Added Pattern Guidance)

```

"Refactor using the Strategy Pattern. Create a BaseQuestionHandler class with
subclasses
  for MCQ, TrueFalse, DragDrop, and ShortAnswer. Separate scoring from UI and
storage.
  Use a Factory to resolve the correct handler by type."

```

Result: Copilot generated the four handler subclasses correctly and wired up a factory function. However, QuizStorageService still called localStorage directly — making it untestable in Node or React Native environments.

Prompt 3 — Final Refinement (Closed the Gap)

```

"The QuizStorageService still hard-codes localStorage. Refactor it to accept a
storage

```

```
adapter via constructor injection so the same service works in browser, Node, and
React Native by swapping adapters. Also add a private #handlers registry with a
fluent registerHandler() method to follow the Open/Closed Principle."
```

Result: Final architecture with full adapter-based storage, private registry, and fluent registration API — environment-agnostic and infinitely extensible.

3. Enhanced Code — Strategy, Factory & Adapter Architecture (V6)

THEME CONFIG — UI DECOUPLED

```
const ThemeConfig = { colors: { success: '#007600', error: '#B12704' } };
```

TELEMETRY SERVICE

```
class TelemetryService {
  logInfo(message, context = {}) { console.info('[INFO]', message, context); }
  logError(message, context = {}) { console.error('[ERROR]', message, context); }
}
```

STRATEGY IMPLEMENTATIONS — SCORING LOGIC ONLY

```
class BaseQuestionHandler {
  execute(questionData, userResponse) { return this.evaluate(questionData,
userResponse); }
  calculateScore(isCorrect, points) { return isCorrect ? points : 0; }
}

class MCQHandler extends BaseQuestionHandler {
  evaluate({ correctAnswer, points }, userResponse) {
    const isCorrect = correctAnswer === userResponse;
    return { score: this.calculateScore(isCorrect, points), isCorrect };
  }
}

class TFHandler extends BaseQuestionHandler {
  evaluate({ correctAnswer, points }, userResponse) {
    // FIX: .trim() prevents failures from accidental whitespace
    const normalized = String(userResponse).trim().toLowerCase();
    const isCorrect = normalized === String(correctAnswer).trim().toLowerCase();
    return { score: this.calculateScore(isCorrect, points), isCorrect };
  }
}

class DragDropHandler extends BaseQuestionHandler {
  evaluate({ correctOrder, points }, userResponse) {
    if (!Array.isArray(userResponse)) return { score: 0, isCorrect: false };
  }
}
```

```

        const matches    = userResponse.filter((item, i) => item ===
correctOrder[i]).length;
        const isCorrect = matches === correctOrder.length;
        const score      = isCorrect ? points : Math.round((matches /
correctOrder.length) * points);
        return { score, isCorrect };
    }
}

class ShortAnswerHandler extends BaseQuestionHandler {
    evaluate({ acceptedAnswers, points }, userResponse) {
        if (typeof userResponse !== 'string') return { score: 0, isCorrect: false };
        const isCorrect = acceptedAnswers.includes(userResponse.trim().toLowerCase());
        return { score: this.calculateScore(isCorrect, points), isCorrect };
    }
}

```

STORAGE ADAPTERS — ENVIRONMENT-AGNOSTIC (FIX APPLIED)

```

// Adapter interface — swap this to support any environment
class LocalStorageAdapter {
    setItem(key, value) { localStorage.setItem(key, value); }
}

class InMemoryAdapter {
    constructor() { this._store = {}; }
    setItem(key, value) { this._store[key] = value; }
}

class QuizStorageService {
    // FIX: accepts any adapter — no hard localStorage dependency
    constructor(telemetry, storageAdapter = new LocalStorageAdapter()) {
        this.telemetry = telemetry;
        this.adapter    = storageAdapter;
    }
    saveScore(score) {
        try {
            this.adapter.setItem('last_score', String(score));
            this.telemetry.logInfo('Score saved', { score });
        } catch (error) {
            this.telemetry.logError('Storage failure', { score, error: error.message });
        }
    }
}

```

UI LAYER

```

class DOMRenderer {
    render({ text, color }) {
        const el = document.getElementById('feedback-box');
        if (el) { el.innerText = text; el.style.color = color; }
    }
}

class QuizUIService {
    constructor(renderer, themeConfig) {

```

```

    this.renderer = renderer;
    this.theme = themeConfig;
  }
  updateFeedback(isCorrect) {
    this.renderer.render({
      text: isCorrect ? 'Correct! Well done.' : 'Try again.',
      color: isCorrect ? this.theme.colors.success : this.theme.colors.error
    });
  }
}

```

APPLICATION SERVICE — PRIVATE REGISTRY + OPEN/CLOSED PRINCIPLE

```

class QuizService {
  #handlers = new Map(); // private — never exposed directly

  constructor({ uiService, storageService, telemetry }) {
    this.uiService = uiService;
    this.storageService = storageService;
    this.telemetry = telemetry;
  }

  registerHandler(type, handler) {
    this.#handlers.set(type, handler);
    return this; // fluent API
  }

  handleQuizAnswer(questionData, userResponse) {
    const timestamp = new Date().toISOString();
    try {
      const handler = this.#handlers.get(questionData.type);
      if (!handler) throw new Error(`No handler for: ${questionData.type}`);
      const result = handler.execute(questionData, userResponse);
      this.uiService.updateFeedback(result.isCorrect);
      this.storageService.saveScore(result.score);
      return { questionId: questionData.id, score: result.score,
        status: result.isCorrect ? 'completed' : 'failed',
        error: false, timestamp };
    } catch (error) {
      this.telemetry.logError('QuizService failure',
        { id: questionData?.id, msg: error.message });
      return { questionId: questionData?.id || null, score: 0,
        status: 'error', error: true,
        message: error.message, timestamp };
    }
  }
}

```

BOOTSTRAP — COMPOSITION ROOT

```

const telemetry = new TelemetryService();

const quizService = new QuizService({
  uiService: new QuizUIService(new DOMRenderer(), ThemeConfig),
  storageService: new QuizStorageService(telemetry, new LocalStorageAdapter()),
  // swap to InMemoryAdapter() for Node/React Native — zero engine changes

```

```
telemetry
});

quizService
    .registerHandler('MULTIPLE_CHOICE', new MCQHandler())
    .registerHandler('TRUE_FALSE',    new TFHandler())
    .registerHandler('DRAG_AND_DROP',  new DragDropHandler())
    .registerHandler('SHORT_ANSWER',   new ShortAnswerHandler());
```

4. Productivity Observations

- Speed: Copilot generated all four handler subclasses (MCQ, TF, DragDrop, ShortAnswer) in under 60 seconds..
 - Iteration quality: The first two prompts produced plausible but incomplete code (coupled storage, no adapter layer). Only the third prompt — targeting the specific remaining flaw — produced a fully environment-agnostic result.
-